Caprice at PCRE2 speed

Measured performance follow-up | Rust experiment v0.2.0 | Anonymous author

Warm matching is now competitive. The new JIT has lower median latency than PCRE2 10.49 on 8/11 original workloads, with a 1.32x geometric-mean throughput advantage. Its latency is 7.45x lower than the submitted PE engine rerun on this host.

PCRE2 uses its direct JIT API. Both engines reuse match storage; baseline capture predicates use borrowed strings. Times include the matching call and any required host predicate. Lower nanoseconds and ratios are better.

Workload	Submitted PE ns/match	New JIT ns/match	PCRE2 10.49 ns/match	JIT / PCRE2
Literal	18.2	15.8	30.3	0.52x
Identifier	459.8	39.0	44.3	0.88x
Key/value fields	567.0	41.8	37.9	1.10x
Alternation	80.0	20.2	39.0	0.52x
Short backreference	169.0	29.9	36.3	0.82x
Long backreference	3,237.5	122.4	249.5	0.49x
Static parameter	44.9	26.1	31.9	0.82x
Lambda guard	644.8	38.6	37.9	1.02x
Dynamic interpolation	325.2	35.2	52.0	0.68x
Length guard	680.0	38.3	35.4	1.08x
Constant lambda	210.2	27.5	38.1	0.72x

The remaining original-suite gaps are fields (1.10x), the length guard (1.08x), and the lambda guard (1.02x). These are median estimates, not significance claims.

The result applies to repeated matching after compilation. JIT compilation and some ambiguous patterns remain substantial gaps. The submitted manuscript and its original measurements remain the historical baseline.

Where parity holds, and breaks

17 additional synthetic workloads: 9 lower medians; 10 within 10% of PCRE2. Geometric-mean latency is 2.2% higher than PCRE2.

Additional workload	New JIT, ns	PCRE2, ns	JIT / PCRE2
16-byte literal	16.6	27.6	0.60x
128-byte literal	22.3	29.8	0.75x
512-byte literal	26.6	42.3	0.63x
Overlapping alternatives	37.6	29.4	1.28x
Disjoint alternatives	34.1	39.4	0.86x
Eight captures	73.0	58.7	1.24x
Backreference, no delimiter	174.5	59.1	2.95x
Long backref, varied failures	111.1	153.8	0.72x
Lazy repeat with delimiter	53.0	51.0	1.04x
Repeated capture	69.1	39.9	1.73x
Guard over long capture	99.5	67.9	1.46x
Interpolation, varied lengths	53.0	60.9	0.87x
Wide character class	96.3	159.8	0.60x
Negated character class	99.6	75.1	1.33x
1 KiB class scan	664.8	775.6	0.86x
Length guard, longer captures	115.4	87.9	1.31x
Captures across alternatives	30.4	36.2	0.84x

The added corpus was generated after the implementation was frozen; no further matcher changes were tuned against these results. It tests longer spans, earlier failures, overlapping alternatives, repeated captures and backreferences without separators. It is still synthetic.

The raw workload ID class_scan_4096 is historical: its generated subjects are actually 1,024-1,280 bytes, including the appended failure byte. The table labels the measured data accordingly.

What changed in the engine

Reusable match state

A caller-owned Scratch retains capture, continuation and undo buffers. All 28 tested workloads perform zero heap allocations in a warmed JIT call using this API; arbitrary expressions can still allocate in the fallback evaluator.

Compact backtracking and capture restoration

A class repetition records one range of candidate endpoints. A capture undo log replaces full capture-vector copies. The range preserves greedy/lazy order, and every resumed candidate restores the same capture environment.

Eliminate provably doomed choices

Disjoint following bytes or absolute end make certain class repetitions possessive. Disjoint first-byte alternatives dispatch directly. A necessary-literal filter rejects absent-literal failures before complex unbounded searches.

Compile capture operations

Length guards, selected equality tests, and ASCII upper/lower interpolation operate on borrowed capture slices. Recognized applied lambdas retain strict evaluation and byte-limit checks. Unsupported expressions keep the original evaluator.

Generate native operations

Cranelift now emits bounded literal loads, static class tests, scanner loops, anchors and branch dispatch. Capture writes, backreferences and lowered expressions use dedicated helpers without opcode dispatch. Other instructions keep the checked interpreter helper.

Correctness evidence

14 Rust tests pass, including the original 36,391 bounded cases, 51,200 expression comparisons, 54,684 capture-trail/reuse comparisons, 11,056 native boundary cases and 15,120 finite-budget comparisons. Release Clippy passes with warnings denied. PCRE2 10.49 passes its four upstream test groups; capture differential tests also pass against it.

The new Rust optimizations are tested and accompanied by explicit semantic obligations. The inherited Lean artifact verifies the earlier expression-specialization kernel; it is not a proof of this compiler, runtime, or native backend.

The next engineering targets

1. Propagate capture-length constraints

Highest-priority warm-match gap: the delimiter-free backreference is 2.95x slower. In a proven full-match shape x followed by its own backreference, the input length fixes 2|x| = |input|. Derive admissible lengths before enumerating endpoints. Likewise, len(x) <= k and equality to a constant can bound or reject captures early. Preserve missing/open capture semantics and strict errors.

2. Inline capture state and factor shared prefixes

Repeated captures are 1.73x slower, eight captures 1.24x, and overlapping alternatives 1.28x. Use an explicit stable capture-slot layout for native writes, reduce unnecessary trail entries using liveness, and factor shared literal prefixes while preserving alternative priority and capture effects.

3. Select scanner and guard code by shape

Long guards and negated classes remain 1.31-1.46x slower. Compare vectorized common-class scanners and fused capture predicates with the existing Rust scanner. The PE interpreter is faster than the JIT on seven of the additional workloads; the compiler should use that evidence when selecting a backend.

4. Tier compilation and reuse compiled programs

Across all 28 workloads, Caprice JIT compilation costs 233-1,958 microseconds, versus 2.6-24.8 microseconds for PCRE2. Keep PE as the initial tier, cache by pattern and static parameters, then compile hot shapes. Where JIT wins, measured compile amortization against PE ranges from about 10,800 to 288,500 matches; seven validation workloads show no break-even.

A concrete next milestone

Bring every measured warm workload within 10% of PCRE2, reduce the largest compile break-even points, and add representative application traffic before making a general performance claim. Required-literal rejection removes the tested missing-b explosion; surviving ambiguous searches can still exhaust their budget.

Measurement and reproduction

Three fresh processes per corpus and PCRE2 version; nine randomized timing samples per workload/engine/process, seven compilation samples. Each corpus workload contains 256 fixed inputs. Timing samples target 20 ms after warmup and calibration. Tables pool 27 timing samples; raw samples, quartiles and process medians are included in the bundle.

The current-version campaign contains 6,669 timing rows and 5,187 compilation rows across the original and additional corpora. The older PCRE2 10.42 campaign and three single-process diagnostic stages are retained separately. Diagnostic stages guided changes; they are not repeated controlled ablations.

Host: AMD EPYC 9V74, Linux x86-64, one pinned logical CPU (0), Rust 1.99.1 release, Cranelift 0.125.3. PCRE2 10.49 was built from its official release tarball using GCC, -O3 -DNDEBUG, shared 8-bit library and JIT enabled. Host load and frequency are not controlled. Shared-container measurements and synthetic inputs limit generalization.

The benchmark uses whole-subject byte semantics and ASCII transforms. PCRE2 postprocessors are timed only for workloads whose full captures are unique; they are not a general substitute for backtracking-aware guards. No Unicode, substring-search, callout, or production-traffic performance equivalence is established.

Reproduce

The bundle contains Cargo.lock, the submitted baseline sources, raw CSVs, corpus generators, source and binary hashes, allocation-counting code, a reviewable patch and the analysis scripts. README.md gives commands to build both versions, select PCRE2 10.49, run the tests and regenerate this note.

Resource and formal scope

Removing failed paths can change finite exhaustion relative to the submitted release. Budget remains distinct from rejection. The optimized VM and JIT are checked for equal residual budgets on the finite test suite. Scans and literal filters are not charged per byte; no wall-clock or global-memory bound is claimed. The native prototype targets 64-bit little-endian hosts and was measured only on x86-64.

Primary references

PCRE2 10.49 release

PCRE2 JIT and direct fast-path API

PCRE2 runtime storage and performance

Cranelift 0.125.3 JIT documentation